LABORATÓRIO DE REDES **PRÁTICA 02**

Do pedido ao cadastro

Consumir uma API REST com Postman e uma interface web

Curso: Engenharia de Controle e Automação · Ifes, Campus Serra
Duração sugerida: 120 minutos de investigação REST, após iniciar o kit; reserve 20–30 minutos para a primeira instalação na EC2.
Organização: duplas/trios, com uma EC2 por equipe e credenciais geradas automaticamente.

O desafio

Você iniciará uma aplicação pronta de inventário de equipamentos fictícios das Plantas A, B e C. Deverá cadastrar, consultar, alterar e excluir recursos; depois, comparar as requisições do Postman com as enviadas por uma interface web.

Aplicação pronta, foco em REST
O repositório contém API, banco, página web, coleção Postman e scripts prontos. Você apenas inicia o kit na sua EC2 e valida o serviço. Não é necessário programar, gerar interface com IA nem receber uma chave do professor. A mesma API será consumida pelo Postman e pelo navegador.

Ao terminar, você deverá conseguir

- Relacionar operações CRUD a métodos, caminhos, corpos e status HTTP.
- Enviar JSON e credenciais no lugar correto; interpretar erros.
- Distinguir substituição completa por PUT de alteração parcial por PATCH.
- Comprovar persistência e isolamento entre grupos.
- Explicar a função do proxy reverso, de HTTPS e de CORS.

Antes de iniciar

Tenha acesso a uma conta AWS autorizada, uma EC2 Ubuntu Server 24.04 LTS dedicada, sua chave SSH e o Postman Desktop na estação. A EC2 pode ser a da prática 1; o kit também funciona em uma instância nova. A conta precisa permitir EC2 e acesso à Internet para baixar o repositório e os pacotes.

Você obtém	Onde encontrar
Repositório e kit	github.com/rafaelrezo/pratica-rest-ec2.
DNS e IPv4 públicos	Console AWS, nos detalhes da sua EC2.
Chaves da API	Geradas pelo script na própria EC2; nunca vêm do professor.
Certificado público	Gerado pelo script e copiado por SSH verificado.

Roteiro: iniciar o kit; validar o acesso; explorar o contrato no Postman; usar a página pronta; comparar as mensagens HTTP. Tudo opera sobre dados fictícios: mudar `ativo` não liga ou desliga uma máquina.

INICIAR / KIT PRONTO

Do repositório à aplicação na EC2

No console AWS: use Ubuntu Server 24.04 LTS em uma instância dedicada, com IPv4 público, DNS público e sub-rede com rota para Internet Gateway. Libere TCP 22 somente para seu IP/rede administrativa e TCP 80/443 para o acesso web; não abra 8000. Se o UFW já estiver ativo, permita OpenSSH e Nginx Full. Confira custos, permissões e o término dos recursos da equipe.

1. Conectar e obter os artefatos

Na estação, substitua os marcadores e confirme a chave do host SSH pelo console EC2 antes de aceitar a primeira conexão:

```
ssh -i SUA_CHAVE.pem ubuntu@SEU_DNS_EC2
```

Na EC2, obtenha o repositório pronto:

```
sudo apt update
sudo apt install -y git
git clone https://github.com/rafaelrezo/pratica-rest-ec2.git pratica-rest-ec2
cd ~/pratica-rest-ec2
```

Sem Git, envie a pasta `kit/` extraída do ZIP por SCP para `~/pratica-rest-ec2/kit/`. Os scripts são os mesmos; não copie trechos de backend do PDF.

2. Iniciar e validar

Ainda na EC2, execute os dois comandos. O início identifica automaticamente o DNS e o IPv4 públicos pelos metadados da instância:

```
bash kit/iniciar-lab.sh
bash kit/validar-lab.sh
```

O primeiro script prepara os serviços, o banco, HTTPS e duas identidades de teste. O segundo mostra método, caminho e status esperado/recebido, criando e removendo apenas seu item de ensaio. **Esperado:** saúde 200, sem chave 401, criação 201, leitura/alteração 200, PUT incompleto 400, isolamento 404, exclusão 204 e consulta final 404.

Validar não encerra a prática
O script confirma que o laboratório está pronto. Nas etapas seguintes, você fará as requisições no Postman e na página para explicar o que cada mensagem significa. Não precisa editar Python, HTML ou configuração do Nginx.

Se repetir a inicialização: as chaves e o banco são mantidos; a página de referência é republicada. Se o DNS/IP mudar ou o certificado expirar, execute novamente e copie o certificado atualizado. Se a descoberta automática estiver desabilitada, use `bash kit/iniciar-lab.sh SEU_DNS_EC2 IP_PUBLICO` com os valores atuais do console. A instalação usa Internet e pode levar alguns minutos.

PREPARAR / ACESSO

Obter sua chave e conferir HTTPS

1. Consultar as credenciais locais

Na sessão SSH privada, fora de gravações/capturas de tela:

```
cat ~/chaves-redes/entrega-grupos.csv
openssl x509 -in ~/redes.crt -noout -fingerprint -sha256
```

O CSV contém g01 e g02, com chaves aleatórias distintas. Use **g01 no CRUD principal**; g02 é uma segunda identidade para testar isolamento na mesma API. Não são duas equipes físicas e não é preciso combinar o teste com colegas. Cada EC2 gera suas próprias chaves, mesmo repetindo esses nomes.

Três arquivos diferentes: `SUA_CHAVE.pem` permite SSH; `redes.crt` é o certificado público HTTPS; a chave de g01 autentica requisições em `X-API-Key`. A chave privada TLS fica no servidor. Não as confunda.

2. Obter somente o certificado público

Em outro terminal, **na estação**:

```
scp -i SUA_CHAVE.pem ubuntu@SEU_DNS_EC2:~/redes.crt .
openssl x509 -in redes.crt -noout -fingerprint -sha256
curl --cacert redes.crt -i https://SEU_DNS_EC2/api/saude
```

Compare as impressões digitais da estação e da sessão SSH. Espere 200 e `{"status":"ok"}`. O certificado autoassinado tem validade de sete dias e cobre o DNS/IPv4 informados. O DNS da AWS não fornece confiança HTTPS automaticamente.

3. Abrir a aplicação pronta

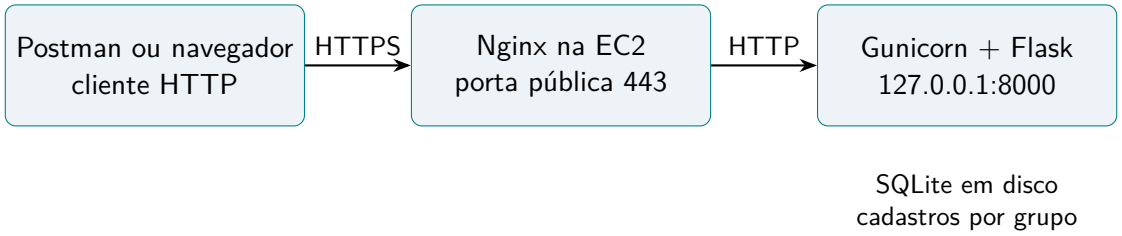
Abra `https://SEU_DNS_EC2/painel.html`. Confira o certificado e estabeleça a confiança específica no navegador. Digite a chave de g01 e clique em "Conectar / listar". A página deve mostrar uma lista vazia ou os registros já criados nessa identidade. A chave fica apenas na memória da página.

Se o acesso externo falhar: o script já testou aplicação e HTTPS dentro da EC2. Confira DNS/IP atuais, regras 443, rota pública e rede da estação. Erro de certificado exige conferir SAN, data e confiança. Não desligue a verificação TLS globalmente nem exponha 8000.

Ponto de partida
Anote a URL, o GET de saúde e o resultado da validação automática, sem registrar chaves. O restante da prática investiga a API que já está funcionando.

ENTENDER A ARQUITETURA

Uma API, vários clientes



O Nginx entrega a página em / e encaminha /api/ para a aplicação. Gunicorn executa o aplicativo Flask como um serviço. SQLite persiste os dados em arquivo. A porta 8000 escuta apenas em loopback e não é liberada no Security Group.

O que torna esta interface orientada a recursos?

`/api/equipamentos` identifica uma coleção. `/api/equipamentos/7` identifica um item. Os caminhos usam substantivos e os métodos expressam ações. As representações são JSON. As respostas utilizam status HTTP e cada requisição protegida leva a credencial necessária.

Este é um CRUD didático inspirado no estilo REST. Usar JSON e métodos HTTP, isoladamente, não demonstra todas as restrições de uma arquitetura REST completa. O objetivo da aula é aplicar semântica HTTP, recursos e representações de forma consistente.

Como o grupo é identificado?

O cabeçalho `X-API-Key` contém a chave de g01 gerada na sua EC2. O servidor associa essa chave ao grupo e restringe todas as operações aos seus registros. Não existe campo `grupo` no corpo; tentar editar um item de outro grupo retorna 404.

Limites: a chave identifica o grupo, não uma pessoa. Trata-se de autenticação didática com autorização por grupo, sem gestão individual de usuários. Ela é enviada por HTTPS; a interface do aluno não consegue esconder de seu próprio usuário a chave que utiliza. Não a publique no código nem em prompts.

Pense antes do primeiro teste
Quem inicia a conexão? Qual programa termina TLS? Quem lê e grava SQLite? Por que CORS não substitui a chave? Identifique também onde os dados ainda podem aparecer em texto claro: cliente, Nginx, loopback e processo da aplicação.

Fonte: Flask: execução com Gunicorn.

CONTRATO / DADOS

Conheça o recurso antes de enviar

```
{
  "id": 7,
  "nome": "CLP-A-01",
  "tipo": "clp",
  "local": "Planta A",
  "ativo": true
}
```

Campo	Regra desta API
`id`	Gerado pelo servidor. Usar na URL; não enviar no corpo.
`nome`	Texto com 1–80 caracteres após retirar espaços das pontas; único por grupo.
`tipo`	Um de: `sensor`, `clp`, `ihm`, `atuador`, `outro`.
`local`	Texto com 1–80 caracteres. Ex.: Planta A, Operação, IDMZ.
`ativo`	Booleano JSON: `true` ou `false`, sem aspas.

Rotas disponíveis

Método	Caminho após `/api`	Resposta de sucesso
GET	`/saude`	200 `{"status":"ok"}`, sem chave.
GET	`/equipamentos`	200 com `itens` e `total`.
GET	`/equipamentos/{id}`	200 com um objeto.
POST	`/equipamentos`	201 com objeto e `Location`.
PUT	`/equipamentos/{id}`	200 com objeto substituído.
PATCH	`/equipamentos/{id}`	200 com objeto atualizado.
DELETE	`/equipamentos/{id}`	204, sem corpo.

Todas as rotas, exceto saúde e preflight OPTIONS, exigem chave. Use caminhos sem barra final. Não há filtros, paginação nem busca no servidor. POST e PUT exigem os quatro campos editáveis; PATCH aceita um ou mais. Campos desconhecidos são rejeitados. O limite de corpo é 16 KiB.

Exemplo de listagem vazia: `{"itens":[],"total":0}`. O ID 7 é apenas ilustrativo. Não presuma que o primeiro cadastro receberá ID 1: os IDs são gerados pela API e compartilhados pelas identidades g01/g02 dessa instância.

POSTMAN / PREPARAR

Configurar ambiente e confiança TLS

1. Abra o Postman Desktop. Pelo botão *Import*, carregue a coleção: `redes-crud.postman_collection.json`; depois, o ambiente: `redes.postman_environment.json`. Ambos estão em `kit/postman/`.
2. Selecione o ambiente "Redes | preencher localmente". Defina `baseUrl` com a URL HTTPS da sua EC2, **sem** `/api` e sem barra final.
3. Defina `apiKey` com a chave de g01 do seu CSV, como valor local/privado ou em um Vault, conforme a versão disponível. Marcar como segredo oculta a exibição, mas não assegura, sozinho, que o valor nunca seja sincronizado. Não compartilhe ambiente preenchido.
4. Deixe `equipamentoId` vazio; a requisição de criação vai preenchê-lo usando o ID da resposta.
5. Se houver certificado público válido, mantenha a verificação TLS padrão. Se autoassinado, obtenha `redes.crt` por canal verificado e adicione-o em *Settings* → *Certificates* → *CA Certificates* no aplicativo Desktop. Verifique SAN e validade. Mantenha *SSL certificate verification* ligada.

Fonte: Postman: Desktop e agentes.

Faça o primeiro GET manualmente

Crie uma requisição, selecione GET e informe:

```
{{baseUrl}}/api/saude
```

Clique *Send*. A resposta deve ser 200, com `{"status":"ok"}`. Esse teste verifica que Nginx e aplicação respondem pela rede, mas ainda não testa sua chave nem uma operação de banco.

Prepare a autenticação das chamadas protegidas

Na coleção importada, a aba *Authorization* já usa *API Key*: chave `X-API-Key`, valor `{{apiKey}}`, local *Header*. Requisições herdam essa configuração; os testes sem chave a desativam de propósito.

Para uma requisição nova fora da coleção, configure manualmente o cabeçalho `X-API-Key: SUA_CHAVE`. Nunca adicione a chave à query string. Ao listar `/api/equipamentos`, o resultado deve ser 200. Sem chave, deve ser 401.

Verificação 1
Registre: sucesso de saúde; tentativa sem chave; listagem autenticada. O relatório deve ocultar o valor de `X-API-Key`. Se saúde responde 200 e listar retorna 401, explique qual parte da infraestrutura já demonstrou funcionar.

CRUD / CREATE E READ

Criar e conferir um cadastro

1. POST para a coleção

No Postman: método POST; URL `{{baseUrl}}/api/equipamentos`; aba *Body* → *raw* → *JSON*. O cabeçalho `Content-Type: application/json` deve estar presente.

```
{
  "nome": "CLP-A-01",
  "tipo": "clp",
  "local": "Planta A",
  "ativo": true
}
```

Esperado: 201 Created; objeto com ID gerado; cabeçalho `Location: /api/equipamentos/ID`. Se o nome já existir em seu grupo, altere-o ou use um cadastro que você ainda não criou.

2. GET para o item criado

Copie o ID real retornado para a variável `equipamentoId`. Na coleção automática, o teste do POST faz isso por você.

```
GET {{baseUrl}}/api/equipamentos/{{equipamentoId}}
X-API-Key: {{apiKey}}
```

O resultado deve ser 200 com os dados enviados e o ID. Faça também GET na coleção e confirme que `itens` contém o cadastro e que `total` corresponde à quantidade retornada.

3. A mesma ideia com curl, opcional

Para comparar ferramentas, configure na estação uma variável temporária, sem colocar a chave no histórico. Use um shell Bash compatível:

```
BASE_URL='https://SEU_DNS_EC2'
read -rsp 'Chave do grupo: ' API_KEY; echo
curl --cacert redes.crt -i "$BASE_URL/api/equipamentos" \
  -H "X-API-Key: $API_KEY"
unset API_KEY
# redes.crt foi obtido pelo canal verificado da pratica 1.
```

O cabeçalho será enviado ao servidor; mantenha o terminal privado. O `curl` e o Postman podem representar a mesma requisição. Não é necessário enviar pelo navegador para usar HTTP.

Verificação 2
Mostre que criar e ler são requisições diferentes. Identifique por que o POST recebe JSON no corpo e por que o ID é obtido da resposta. Explique por que repetir o POST pode produzir 409, enquanto repetir o GET não cria outro equipamento.

CRUD / UPDATE

PUT e PATCH não pedem a mesma coisa

1. PATCH: alterar apenas um campo

Método PATCH na URL do item. Corpo JSON:

```
{"ativo": false}
```

Esperado: 200; `ativo` fica falso; nome, tipo e local permanecem. Leia novamente por GET para conferir. O campo apenas informa o estado do cadastro.

2. PUT: enviar toda a representação editável

Método PUT na mesma URL. Corpo JSON:

```
{
  "nome": "CLP-A-01",
  "tipo": "clp",
  "local": "Planta B",
  "ativo": true
}
```

Esperado: 200, com todos os campos fornecidos. Neste contrato, PUT substitui um item existente; não cria um ID desconhecido. Se o item não existir, retorna 404. Esse comportamento está definido pela API e não deve ser adivinhado pelo cliente.

3. Teste a diferença de propósito

Envie somente `{"ativo":false}` por PUT. A API deve responder 400, pois faltam campos obrigatórios. Envie o mesmo corpo por PATCH: deve responder 200. Não generalize que todo backend valida de forma idêntica; o cliente deve seguir o contrato recebido.

4. Faça uma previsão antes de executar

Operação	Previsão de efeito	Evidência após o GET
PATCH `ativo=false`	_____	_____
Mesmo PATCH novamente	_____	_____
PUT completo	_____	_____
PUT incompleto	_____	_____

Verificação 3
Justifique por que atribuir o mesmo valor com PATCH pode ser idempotente. Explique também por que o corpo de PUT inclui `ativo`, mesmo quando você só pretendia alterar `local`.

CRUD / DELETE E ERROS

Resultados esperados também incluem falhas

1. DELETE do cadastro criado pelo grupo

Envie DELETE para `/api/equipamentos/ID`. **Esperado:** 204 No Content, sem corpo. Depois, GET para o mesmo ID deve retornar 404. Repetir DELETE também retorna 404; o recurso continua ausente.

2. Execute estes testes controlados

Alteração no teste	Status	Explicação esperada
Retirar a chave de uma rota protegida	401	Não foi apresentada credencial válida.
POST incompleto ou JSON inválido	400	A representação recebida é inválida.
Enviar `ativo: "true"` no JSON	400	Texto não é booleano.
Repetir nome no mesmo grupo	409	Conflito com recurso existente.
Enviar `text/plain` no POST	415	Formato de corpo não aceito.
PUT na coleção, sem ID	405	Método não disponível nesse recurso.
GET de item excluído	404	Esse item não está disponível no grupo.

Erros da aplicação têm corpo `{"erro":"mensagem"}`. Erros do Nginx, como 502, podem vir em HTML. Por isso, o cliente precisa verificar status e conteúdo; não deve presumir que toda resposta sempre é JSON.

3. Isolamento sem depender de outra equipe

Crie um novo item com a chave de g01 e anote o ID. Duplique o ambiente Postman e use nele a chave de g02 do seu CSV, mantendo `baseUrl`. Para o ID de g01, GET, PATCH, PUT e DELETE com g02 devem retornar 404. Volte a g01: GET deve retornar 200 e os dados devem estar intactos. Exclua o item com g01. A segunda chave é válida; o item é que está fora de seu escopo.

Verificação 4
Preencha uma tabela com método, caminho, status, corpo resumido e interpretação para criar, listar, consultar, alterar, excluir e dois erros. Não inclua chaves. Uma resposta 404 é evidência de que o servidor respondeu HTTP; ela não equivale a timeout de rede.

Limite de segurança do exercício: a API possui validação e isolamento por grupo, mas não foi projetada para uso de produção. Não utilize dados pessoais nem a conecte a CLPs, sensores ou atuadores reais.

POSTMAN / REPETIR

Executar a coleção e ler os testes

Sequência da coleção fornecida

Ordem	Teste	Status esperado
01–03	Saúde; listagem sem chave; listagem autenticada.	200, 401, 200.
04–06	Criar; ler o ID; repetir o nome.	201, 200, 409.
07–08	PATCH parcial; PUT completo.	200, 200.
09–11	Dados incompletos; formato errado; método errado.	400, 415, 405.
12–13	Excluir; ler o excluído.	204, 404.

Execute os itens em ordem, manualmente ou em *Run Collection*, conforme os limites do plano. O POST usa um nome com instante atual para reduzir colisões entre execuções e salva o ID real em `equipamentoId`. Não rode duas execuções concorrentes no mesmo ambiente, pois elas compartilham essa variável.

Entenda um teste de resposta

Na aba *Scripts* → *Post-response* ou *Tests*, conforme sua versão:

```
pm.test("Criado", () => pm.response.to.have.status(201));
const dados = pm.response.json();
pm.environment.set("equipamentoId", String(dados.id));
```

Esse script é executado depois da resposta. Uma asserção aprovada verifica algo delimitado, como status ou campo. Não comprova que toda a infraestrutura é segura ou que todos os clientes funcionarão.

O caso 204 exige tratamento próprio

```
pm.test("Excluído sem corpo", () => {
  pm.response.to.have.status(204);
  pm.expect(pm.response.text()).to.eql("");
});
```

Não execute `pm.response.json()` indiscriminadamente para 204: não há JSON para interpretar. A interface web enfrenta o mesmo problema se chamar `response.json()` sempre.

Verificação 5
Entregue o resultado da execução, com os 13 itens e status esperados. Se um teste falhar, registre qual pré-condição não foi atendida. Um teste que espera 401 e recebe 401 deve passar; o objetivo não é obter 200 em todas as chamadas.

APLICAÇÃO WEB / CLIENTE PRONTO

A página consome a mesma API

A interface já foi publicada pelo script em `/painel.html`. Não há etapa de programação nem build. O código JavaScript do navegador envia requisições HTTP para a mesma API que você explorou no Postman.

1. Abra `https://SEU_DNS_EC2/painel.html`, informe a chave de g01 e liste os cadastros.
2. Abra DevTools → Network/Rede. Limpe a lista e crie um cadastro pela página. Localize POST `/api/equipamentos`: observe JSON, `Content-Type`, status 201 e o ID retornado. Oculte a chave nas evidências.
3. Consulte esse mesmo ID pelo Postman com g01. Deve retornar os mesmos dados. A interface não possui um inventário separado.
4. Altere `ativo` pelo Postman com PATCH. Liste novamente na página e confira a mudança. Faça o inverso: altere na página e confira por GET no Postman.
5. Exclua pela página com confirmação. Localize DELETE e o 204 sem corpo na aba Network; consulte o ID no Postman e confira 404.

Do clique à representação

Ação na página	Requisição observada	Evidência na resposta
Conectar / listar	GET da coleção	Lista JSON e total, 200.
Cadastrar	POST da coleção	ID do novo recurso, 201.
Ler um item	GET com ID	Representação do recurso, 200.
Salvar edição completa	PUT com ID	Representação atualizada, 200.
Alternar ativo	PATCH com ID	Campo alterado, 200.
Excluir	DELETE com ID	Corpo vazio, 204.

Verificação 6
Escolha uma operação e explique a sequência: ação do usuário; método/URL/corpo enviados; resposta da API; atualização da tela. Compare o cadastro pelos dois clientes. O que permite criar outro aplicativo para a mesma API sem reescrever o armazenamento?

Por que isso permite construir aplicações web? A API oferece um contrato de recursos e representações. Clientes distintos utilizam esse contrato por HTTP; a persistência e as regras de acesso permanecem no servidor. A página transforma ações do usuário em requisições e respostas em elementos visuais. REST orienta essa organização; não exige uma interface específica.

Aprofundamento opcional: o kit preserva o prompt completo do v0 no anexo, para quem quiser criar outra interface depois. Isso não é requisito de execução nem de entrega desta prática.

NAVEGADOR / ORIGENS

Evitar dois problemas comuns de integração

Mesma origem: o caminho recomendado

Uma origem é a combinação de esquema, host e porta. Se a página está em `https://SEU_DNS_EC2` e chama `/api/equipamentos`, a requisição usa a mesma origem. O Nginx encaminha internamente para o Python. A porta 8000 não aparece na URL pública e não é aberta para os alunos.

Origem diferente: o caso da prévia do v0

Uma página em outro domínio que faça `fetch` para a EC2 está em outra origem. O navegador exige uma política CORS compatível. Com JSON e `X-API-Key`, normalmente envia antes OPTIONS (preflight), consultando método e cabeçalhos permitidos. A aplicação já sabe responder, mas uma origem externa só será aceita se cadastrada em `LAB_CORS_ORIGINS`. O caminho desta prática usa a mesma origem e dispensa esse ajuste.

A chave continua necessária. CORS é uma política aplicada por navegadores; não substitui autenticação nem um firewall. O Postman Desktop e o `curl` não aplicam a mesma restrição de origem que uma página. Consequentemente, funcionar no Postman não prova que funcionará no navegador. Fonte: MDN: CORS e preflight.

HTTPS chamando HTTP: conteúdo misto

Se a página é HTTPS e tenta chamar uma API HTTP remota, o navegador pode bloquear a requisição como conteúdo misto. Liberar CORS não corrige isso. Use HTTPS na API ou sirva a página e a API pela mesma origem HTTPS no Nginx. Fonte: MDN: conteúdo misto.

Certificado autoassinado em outra origem: uma prévia hospedada pode falhar por confiança TLS, por CORS ou por ambos. Para a aula, teste o site exportado e a API juntos na EC2. Se a prévia externa for indispensável, use certificado publicamente confiável e permita somente a origem exata da página, após inspecionar o cabeçalho `Origin`. A origem pode mudar entre prévias.

Não são correções válidas
Não usar `mode: 'no-cors'`: a resposta fica opaca para o JavaScript e os cabeçalhos necessários são limitados. Não desabilitar a segurança do navegador e não expor a porta 8000 para contornar o proxy.

Teste de distinção

A API funciona no Postman, mas a página mostra "Failed to fetch". Verifique primeiro Console, Network, URL, certificado, esquema e OPTIONS. Se o servidor responde 401 em JSON, a requisição chegou à aplicação: investigue a chave.

DIAGNÓSTICO E ENTREGA

Transformar observação em explicação

Sintoma	O que observar	Próximo teste
Timeout	Não há resposta HTTP.	Rede/SG/rota; comparar com saúde local.
Erro de certificado	Falha antes de HTTP.	Confiança, SAN, data e nome/IP.
502	Nginx responde; upstream falha.	Testar serviço e saúde em `127.0.0.1:8000`.
401	A aplicação rejeita a chave.	Ambiente selecionado e header correto.
400 / 415	Corpo ou formato incorreto.	Conferir JSON, tipos e Content-Type.
404	Caminho/ID/grupo.	Listar os seus itens e usar ID real.
409	Nome duplicado no grupo.	Consultar antes de repetir criação.
Só navegador falha	Console, OPTIONS e TLS.	Investigar CORS e conteúdo misto.
Interface mostra erro após excluir	DELETE retornou 204?	Não fazer parse de corpo ausente.

Persistência e diagnóstico na sua EC2

Crie um item com g01, anote o ID e execute `sudo systemctl restart redes-api` por SSH. Repita GET: espere 200 e os mesmos dados. Se houver 502, use `sudo systemctl status redes-api -no-pager` e `sudo journalctl -u redes-api -n 30 -no-pager`. Compare com a saúde local em `http://127.0.0.1:8000/api/saude`. Exclua o item após registrar a evidência.

Entrega do grupo

1. Tabela de operações com método, caminho, corpo resumido, status e efeito confirmado por GET.
2. Resultado dos 13 testes da coleção ou execução manual equivalente.
3. Comparação fundamentada entre PUT/PATCH, 401/404 e falha de transporte/erro HTTP.
4. Evidência de persistência após reiniciar a API e isolamento entre g01/g02, sem exposição de chaves.
5. URL e captura do Network mostrando uma operação da página pronta sobre o mesmo cadastro acessado no Postman; explique como a interface utiliza o contrato REST.

Perguntas para discussão

Por que POST não é sinônimo de "enviar qualquer coisa"? Por que sucesso no ping não prova o CRUD? Por que a chave não pertence à URL? Por que o Nginx pode ver o JSON mesmo quando o aluno usa HTTPS? Que evidência distingue 502 de 401?

Ao encerrar: exclua somente os cadastros do seu grupo; apague a chave de ambientes compartilhados; feche a interface; remova certificados/exceções didáticas adicionados à estação. Salve as evidências e termine a EC2 da sua equipe quando não for mais utilizada; confira recursos remanescentes e custos. Não há API docente compartilhada neste roteiro.

Escopo: scripts, API, página pronta e coleção acompanham o repositório/ZIP. O guia docente descreve a preparação da distribuição e os critérios de avaliação; nenhuma etapa do aluno depende de ações do professor no servidor.

Revisão do fluxo autônomo em 23/10/2026; referências técnicas de 16–23/10/2026. Os nomes de abas e a disponibilidade de recursos do Postman/v0 variam conforme versão e plano.

ANEXO OPCIONAL / FORA DO NÚCLEO

Criar outra interface com v0

Copie este texto ou use o arquivo editável correspondente do kit. Não acrescente chaves reais. O aplicativo gerado deverá cumprir o contrato do backend fornecido.

```
Evolua o portal para uma interface de inventário didático usando esta API
REST já existente. Não crie outro backend, banco de dados ou mocks que
simulem sucesso. A página será exportada estaticamente e servida no Nginx
da EC2. Continue com output: 'export' e os requisitos da prática 1.

Toda chamada fetch deve ocorrer no navegador (componente cliente).
URL base padrão: /api. Permita informar outra URL HTTPS confiável.
O host público será o DNS da própria EC2 fornecido pela AWS, com certificado
autoassinado didático confiado no cliente. Não fixar esse nome nem um IP
no código; /api acompanha a origem da página.
Campo de senha para digitar a chave do grupo, mantida apenas na memória.
Enviar X-API-Key em cada chamada protegida. Nunca colocar uma chave real
no código, prompt, variável NEXT_PUBLIC, URL, log, localStorage ou Git.
Não solicite credenciais AWS. Não use proxy da Vercel nem Server Actions.

Contrato, caminhos relativos à URL base:
GET /saude: sem chave; 200 {"status":"ok"}.
GET /equipamentos: 200 {"itens":[objeto,...],"total":n}.
GET /equipamentos/{id}: 200 objeto, ou 404.
POST /equipamentos: recebe os 4 campos abaixo, retorna 201 objeto com id
  e cabeçalho Location. ID deve ser lido da resposta, nunca presumido.
PUT /equipamentos/{id}: substitui os 4 campos; retorna 200 objeto.
PATCH /equipamentos/{id}: altera um ou mais campos; retorna 200 objeto.
DELETE /equipamentos/{id}: retorna 204 SEM corpo; não chamar json() nele.
Objeto: {"id":1,"nome":"CLP-01","tipo":"clp",
         "local":"Planta A","ativo":true}.
Campos nome e local: strings de 1 a 80 caracteres, sem apenas espaços.
tipo: sensor | clp | ihm | atuador | outro. ativo: booleano JSON.
Nome único por grupo. Não enviar id no corpo. Não adicionar barra final.
Não existe filtro de busca na API; se necessário filtre no cliente.
Erros da aplicação: {"erro":"mensagem"}. Nginx também pode retornar HTML.
Status 400 validação/JSON; 401 chave inválida;
403 origem não permitida; 404 recurso inexistente; 409 nome duplicado;
413 corpo grande; 415 Content-Type incorreto; 405 método não permitido.

Telas/estados: conexão, tabela, cadastro, edição completa via PUT, alternar
ativo via PATCH, leitura individual via GET e exclusão com confirmação.
Mostrar carregamento, lista vazia, mensagens de erro e status HTTP.
Exibir método, caminho e código da última operação, sem exibir a chave.
Tratar 204 antes de fazer parse. Impedir clique duplicado. Renderizar dados
como texto seguro. Não dizer que mudar ativo liga ou desliga máquinas.

Mesma origem na EC2: /api dispensa configurar CORS adicional.
Preview v0 pode ter outra origem. Não presuma que funcionará: a API precisa
de certificado público confiável e permitir explicitamente essa origem.
Se isso faltar, mostre orientação para testar a versão exportada na EC2.
Não usar mode: 'no-cors' e não desativar segurança do navegador.
Entregue instruções para npm run build e publicação de out/ no Nginx.
```